

Cell-as-Substrate: Cells Holding Coordination State Outside Substrate as Standalone Anti-Pattern in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 6, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize one anti-pattern named in the source paper — **cell-as-substrate**, the deployment configuration in which cells hold coordination state outside the substrate — as a standalone failure mode with independent operational content, separable from the broader commitments it violates and testable on its own terms.

Abstract

The Coordination Knowledge Substrate (CKS) pattern commits to a substrate-cell boundary (the substrate holds state, cells execute behavior) and to substrate-as-source-of-truth (the substrate is the authoritative answer to coordination questions). Both commitments fail simultaneously, through the same operational configuration, when cells hold coordination state outside the substrate — when an LLM-based cell carries forward context across executions, when a service-based cell maintains runtime state between invocations, when a cell caches substrate content and the cache becomes authoritative through accumulated drift, when an agent-based cell maintains "memory" of prior interactions. This note formalizes that failure mode — *cell-as-substrate* — as a standalone anti-pattern. It states the four operational components, identifies the CKS commitments violated and the cascade through path retraceability and determinism, traces the operational failure mode, specifies the architectural correction including the substrate-derived-view correction for LLM-cell context-window needs, distinguishes cell-as-substrate from four adjacent legitimate cell behaviors, and provides an operational test with three sharpening properties.

1. Why the cell-as-substrate anti-pattern needs to be formalized as standalone

The CKS pattern's substrate-cell boundary (§2.1) commits to the architectural separation between substrate (state) and cells (behavior). The substrate-as-source-of-truth commitment (§11.3) commits to the substrate as the authoritative answer to coordination questions across five categories — what was decided, by whom, under what authority, with what rationale, and where contradictions remain. Both commitments are independently formalized in this series. The cell-as-substrate failure mode is named in passing in the source paper (§6.2, under the "context rot" framing) but not given a standalone treatment.

A standalone treatment earns its keep on three grounds. First, cell-as-substrate violates two foundational commitments simultaneously, through the same operational configuration: cells holding state collapses the substrate-cell separation (failing A1.02) and introduces multiple sources of truth for content the substrate should be authoritative for (failing A1.08). Without a standalone treatment, deployments may recognize one violation but not the other.

Second, the failure mode is operationally common in 2024–2026 AI agent frameworks. LLM-based cells naturally accumulate memory through context windows, conversation history, fine-tuning state, or in-cell caching; agent-based cells commonly maintain runtime memory across invocations. The drift is steady because audiences understand "the agent remembers" as a positive capability without recognizing the architectural consequence — that the substrate-cell boundary collapses and the substrate stops being authoritative for content cells hold.

Third, the failure mode cascades. A1.07 (path retraceability) and A1.10 (the determinism contract) both fail through the same hidden-cell-state mechanism: the retraceable trail breaks at cell boundaries because cell-internal state is not substrate-recorded; determinism fails because cell behavior depends on hidden state. Without publishing the anti-pattern as standalone, deployments may not recognize the architectural cascade through these subsequent commitments.

2. The anti-pattern, defined precisely

A deployment exhibits **cell-as-substrate** when cells hold coordination state outside the substrate — when state that should be substrate-resident under A2.43–A2.47 is instead carried in cell-internal storage, persisting across executions or affecting subsequent decisions. The anti-pattern has four operational components.

(a) Cell-internal state persisting across executions. The cell holds state that persists between executions: after one execution completes, cell-internal state remains and affects the next. The persistent state may take the form of LLM context windows that carry coordination content across turns, runtime variables in service-based cells, conversation history in agent-based cells, or accumulated state from prior executions. The architectural commitment per A2.08 — that cells execute behavior on substrate state without retaining it across executions — fails operationally.

(b) Cell-held intermediate results affecting subsequent decisions. The cell holds intermediate results from processing — computed values, lookup results, derived content — and these results affect subsequent decisions. The intermediate results are not recorded in substrate and not visible to other cells or to humans exercising the inspect right; decisions cascade off cell-internal state the substrate cannot expose.

(c) Cell-cached coordination content not in substrate. The cell caches content that is conceptually substrate state — "what is the case" per A2.43, "what is current" per A2.44, "what is in conflict" per A2.45, "what rules apply" per A2.46, "who has what authority" per A2.47 — but maintains the cache locally rather than reading authoritatively from substrate. The cache operates as a parallel store of coordination state with potential for drift.

(d) Cell-internal "memory" structures duplicating substrate-scope state. The cell maintains memory structures — LLM context windows, agent memory stores, conversation history — that hold content the substrate should be authoritative for. The cell's memory operates as a duplicate of substrate

state, with operational pressure for authority to migrate to the cell as the cell's memory becomes the more readily accessible source.

A deployment exhibiting any one component partially exhibits the anti-pattern; one exhibiting all four exhibits it fully. The distinction between cell-as-substrate and acceptable cell-internal state is precise: within-execution ephemeral state — computed values, scratch calculations, runtime variables cleared at execution end — is acceptable per A2.08; cross-execution persistent state is the failure. The commitment is that coordination state lives in substrate, not that cells perform no internal computation.

3. Which CKS commitments are violated

Cell-as-substrate violates two foundational CKS commitments directly and cascades into violations of three more.

A1.02 (the substrate-cell boundary) — directly violated. A1.02 commits to the architectural separation between substrate (which holds state) and cells (which execute behavior). Cell-as-substrate fails this separation by having cells hold state; the boundary collapses architecturally. The decomposition across A2.08–A2.12 is operationally compromised — particularly A2.08's commitment that cells execute behavior without holding state across executions and A2.10's commitment that the substrate holds state.

A1.08 (the substrate is the source of truth) — directly violated. A1.08 commits to the substrate as the source of truth for coordination content. Cells holding coordination state introduce multiple sources of truth, operationally migrating authority from substrate to cells. The source-of-truth decomposition across A2.42–A2.48 is extended-violated category by category: when cells hold "what is the case" content, A2.43 fails; when cells hold "what is current" content, A2.44 fails (the canonical "context rot" instance per §6.2 of the source paper); when cells hold conflict information, A2.45 fails; when cells hold rule state, A2.46 fails; when cells hold authority information, A2.47 fails.

A1.07 (path retraceability) — extended violation. The retraceable trail per A1.07 lives in substrate; cell-internal state is not substrate-recorded, so the trail breaks at cell boundaries. Decisions made based on cell-internal state are not reproducible because the state is not visible. The accountability commitments per A2.36–A2.39 fail operationally because rationale lives in cell-internal state the substrate cannot expose.

A1.10 (the determinism contract) — extended violation. A1.10 requires substrate operations to be deterministic. Cell-as-substrate introduces hidden non-determinism: two seemingly identical substrate states produce different cell behavior because cells' internal states differ. The substrate state alone is no longer sufficient to reproduce cell behavior.

A1.03 (conflict preservation) — operationally compromised. If conflicts are cell-held rather than substrate-recorded, A1.03's preservation commitment operates only for substrate-recorded conflicts; cell-held conflicts are operationally invisible and may be silently resolved or lost.

The two direct violations are conceptually distinct — one is about architectural separation, the other about authoritative content — but operationally produced by the same configuration. The three extended violations follow from the same hidden-cell-state mechanism. A single operational drift produces violations across five commitments.

4. The failure mode

The commitment violations identified in §3 manifest in deployment as five operationally specific consequences.

Source of truth fragments across substrate and cells. Coordination state lives partially in substrate and partially in cells; humans exercising the inspect right see only the substrate-resident portion, missing the cell-held portion. The substrate is no longer authoritative for content the cells hold.

Path retraceability breaks at cell boundaries. The retraceable trail records substrate state changes and cell consultations; cell-internal state is not recorded. A reviewer asking "why did this cell decide as it did" can read the substrate input but not the cell-internal state that influenced the decision.

Cell behavior depends on hidden state. Two seemingly identical substrate states produce different cell behavior because cell-internal state differs. A test that re-runs the cell against the same substrate input gets different results depending on cell-internal state at run time.

Cell state drifts over time. Cell-internal state accumulates over executions; over deployment lifecycle, cell-held content drifts from substrate state. The "context rot" failure mode at §6.2 is one instance: substrate content participants thought authoritative has been compressed, summarized, or re-embedded in cell-held memory and now disagrees with the substrate.

Hidden cell state masks architectural failures and migrates authority operationally. A deployment may appear governance-compliant under substrate inspection while operating inconsistently due to cell-held state — a substrate audit finds substrate content correct, while the cell-held content driving cell decisions is silently drifting. Authority migrates by the same mechanism: even if substrate-resident authority structure per A2.47 specifies humans as authority-holders, the cell's memory becomes the operationally accessible source, and humans exercising rights against substrate find it has lost relevant content to cell-held storage.

5. The architectural correction

The architectural correction operates through three foundational commitments together: A1.02's substrate-cell separation, A1.08's substrate-as-source-of-truth, and A2.08's commitment that cells execute behavior without holding state across executions.

All coordination state in substrate. Coordination state — content in any of the five categories per A2.43–A2.47 — must live in substrate. Cells do not hold this state internally; they read it from substrate at execution start (per A2.19's Property A) and write changes back to substrate at execution end (per A2.20's Property B).

Substrate-cell boundary preservation. Cells execute behavior on substrate state but do not hold that state. Cells do not maintain state across executions, do not cache coordination content locally, and do not maintain memory structures that duplicate substrate-scope state. State that needs to persist between executions must be written back to substrate.

Within-execution state ephemerality. Cell-internal state during a single execution is acceptable when ephemeral — computed values, scratch calculations, runtime variables cleared at execution end. The architectural commitment is that this state does not persist or affect subsequent executions.

The operationally consequential case is LLM-based cells with conversation-history needs. The correction here runs through Pattern B per A2.93: **LLM context as substrate-derived view, not cell-held memory**. For LLM-based cells where conversation history or accumulated context is needed for subsequent reasoning, the relevant content must be substrate-resident. The cell reads relevant substrate content as input per A2.19 at the start of each execution; the LLM's context window is operationally a substrate-derived view (Pattern B: adjacent component as derived view), not a persistent cell-held memory. Outputs are written back to substrate per A2.20. The view is recomputed per execution; the substrate remains authoritative; the LLM's context window is rebuilt from substrate content rather than carried across executions as cell state.

A correctly architected deployment additionally maintains a state-persistence audit (cell behavior depends only on substrate input, not on accumulated cell-internal state) and preserves retraceability across cell boundaries (cell consultations of substrate are recorded per A2.40; decisions based on substrate content are reproducible because the substrate state is visible).

6. What the anti-pattern is NOT

Several adjacent cell behaviors are commonly conflated with cell-as-substrate but do not exhibit it.

Not within-execution ephemeral state. Runtime variables, computed values, or scratch calculations during a single execution are legitimate per A2.08 when cleared at execution end. The failure arises specifically when state persists *across* executions.

Not computed values that don't persist. Intermediate values, transformations, or derived results computed during execution are legitimate when not retained beyond the execution. Computation is the cell's job; persistence beyond the execution is the failure.

Not scratch calculations cleared at execution end. Temporary buffers, working memory, and intermediate states needed for the cell's reasoning are legitimate when cleared. Scratch calculations that persist into the next execution exhibit the anti-pattern; scratch calculations that do not persist do not.

Not cell-runtime variables that don't affect coordination. Cells may carry runtime variables for non-coordination concerns — debugging state, performance counters, logging buffers — without exhibiting the anti-pattern. Variables holding coordination-relevant state in any of the five categories per A2.43–A2.47 exhibit the anti-pattern; variables for non-coordination concerns do not.

The distinction across all four cases is the same: persistence of coordination-relevant state across executions is the failure; everything else is the cell doing its job.

7. Operational test

A deployment exhibits cell-as-substrate if any of the following hold at any time during the deployment's existence.

1. Cells maintain state across executions; cell-internal state persists between executions and affects subsequent decisions.
2. Cells hold content in any of the five categories of substrate-authoritative state per A2.43–A2.47 in cell-internal storage rather than reading authoritatively from substrate.

3. Cell behavior depends on cell-internal state not visible in substrate; two cells reading the same substrate state produce different outputs because cell-internal state differs.
4. LLM-based cells maintain context windows or memory structures holding coordination content across executions without that content being substrate-resident and without the context window being recomputed per execution from substrate.

Three sharpening properties operationalize the test for deployment review.

(a) State-persistence test. Verify that cell-internal state does not persist across executions. Cell state at execution start should depend only on substrate input, not on prior execution's cell-internal state. A cell re-executed against the same substrate input from a clean cell-internal start should produce the same output as a cell continuing from prior executions.

(b) Retraceability-across-cells test. Verify that decisions made by cells are reproducible from substrate alone. Re-execute cells with identical substrate state; outputs should be consistent with the determinism contract per A1.10. Where outputs differ, the cell-internal state responsible localizes the cell-as-substrate violation.

(c) Source-of-truth-fragmentation test. Verify that coordination state lives in substrate, not in cells. Inspect cell-internal storage for content in any of the five categories per A2.43–A2.47. Where such content is found in cells, it is substrate-scope state held outside substrate, and the deployment exhibits cell-as-substrate for that category.

A deployment that fails any of (1)–(4) and any of (a)–(c) exhibits the anti-pattern; the architectural correction per §5 specifies the operational changes required. *One-sentence test:* if a deployment's cells maintain state across executions — through LLM context windows that persist, agent memory structures, cached coordination content, or runtime variables holding substrate-scope state — and cell behavior depends on this cell-internal state rather than only on substrate input, the deployment exhibits cell-as-substrate; A1.02 and A1.08 both fail, with A1.07 and A1.10 cascading.

8. Conclusion

Cell-as-substrate is the failure mode where the substrate-cell boundary collapses and the substrate stops being the source of truth, both at once. The two violations are conceptually distinct but operationally produced by the same configuration: cells holding coordination state outside substrate. The cascade through path retraceability and the determinism contract follows from the same hidden-cell-state mechanism — cell decisions depend on state the substrate cannot expose; the trail breaks at cell boundaries; behavior becomes non-reproducible from substrate input alone.

The anti-pattern is operationally common because LLM-based and agent-based cells naturally accumulate memory through context windows, conversation history, fine-tuning state, and in-cell caching. The drift is steady because audiences understand "the agent remembers prior interactions" as a positive capability without recognizing the architectural consequence. Naming the anti-pattern as standalone — with the four operational components, the joint and cascading violations, the failure mode, the architectural correction including the substrate-derived-view treatment for LLM context, the four adjacent-pattern distinctions, and the operational test with three sharpening properties — gives downstream readers a precise specification of the failure mode and its correction.

This is the first of three substrate-cell-boundary anti-patterns in this series. Subsequent notes formalize substrate-as-cell (the inverse failure: substrate executing behavior) and cell-to-cell direct communication (cells passing state to each other outside substrate). Subsequent work that adopts the CKS pattern, extends it, composes it with adjacent patterns, or argues against it should treat cell-as-substrate as the joint A1.02 + A1.08 failure mode formalized here. Subsequent work that uses the term differently, or that fails to recognize the joint and cascading character of the violation, is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Cell-as-Substrate: Cells Holding Coordination State Outside Substrate as Standalone Anti-Pattern in the Coordination Knowledge Substrate Pattern*. May 6, 2026. ORCID: 0009-0004-8065-3235.